SLOT 9

LAYOUT & NAVIGATION NÂNG CAO

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- iOS URL Scheme Navigation
- Flex Layout (Row/Column)
- GridView & Stack

1 Flex Layout với `Row` & `Column` (Tư duy bố cục theo trục)

Khái niệm cốt lõi

`Row` và `Column` là **Flex layout**: sắp xếp widget **theo 1 trục chính** (ngang hoặc dọc)

Hiểu đúng:

`mainAxis` → trục sắp xếp chính

`crossAxis` → trục vuông góc

`Expanded`, `Flexible` dùng để **chia không gian động**, không phải để "cho đẹp"

Mục tiêu học tập

Sau bài này, sinh viên có thể:

Xây dựng UI dạng **form**, **profile**, **card thông tin**

Biết **vì sao UI bị tràn màn hình** và cách xử lý

Chủ động điều chỉnh **tỉ lệ – khoảng cách – căn chỉnh** UI

Mô tả sư phạm ngắn

Đây là **nền móng layout Flutter**

Không hiểu Flex → mọi layout phức tạp sau này đều "code mò"

👉 Học để làm gì trong app thật?

→ Sắp xếp avatar + tên + nút action

→ Layout màn hình đăng nhập / đăng ký

→ UI responsive cơ bản trên nhiều kích thước màn hình

2 GridView – Layout dạng lưới (Nhiều phần tử, có cấu trúc)

Khái niệm cốt lõi

`GridView` dùng khi UI **không còn tuyến tính**

Các phần tử được sắp xếp theo **hàng & cột**

Phân biệt:

`GridView.count` → số cột cố định

`GridView.builder` → dữ liệu động (API, danh sách lớn)

Mục tiêu học tập

Sinh viên có thể:

Hiển thị **gallery ảnh**, **danh sách sản phẩm**, **dashboard**

Biết khi nào **không nên dùng ListView**

Tối ưu hiển thị khi dữ liệu nhiều

Mô tả sư phạm ngắn

GridView giúp UI **gọn – trực quan – giống app thương mại điện tử**

Là bước chuyển từ "bài tập sinh viên" → "app đời thật"

👉 Học để làm gì trong app thật?

→ Màn hình danh sách sản phẩm

→ Trang chọn danh mục

→ Hiển thị album ảnh / icon chức năng

PhucPT10@fpt.edu.vn

3 Stack – Layout chồng lớp (UI có chiều sâu)

Khái niệm cốt lõi

`Stack` cho phép **xếp chồng widget lên nhau**
`Positioned` dùng để xác định vị trí tương đối
Quan trọng: **thứ tự widget = thứ tự lớp**

Mục tiêu học tập

Sinh viên có thể:

Tạo UI có **badge**, **icon nổi**, **overlay**
Hiểu vì sao widget "đè lên nhau"
Kiểm soát layout thay vì chỉnh mò padding/margin

Mô tả sư phạm ngắn

Stack tạo **chiều sâu thị giác**, giúp UI giống app thật
Là nền cho animation, floating button, banner

Học để làm gì trong app thật?

→ Avatar có icon online
→ Card có nhãn "SALE"
→ Nút nổi trên bản đồ / ảnh nền

4 iOS URL Scheme Navigation (Kết nối app với hệ điều hành)

Khái niệm cốt lõi

URL Scheme cho phép:

Mở app Flutter từ **link bên ngoài**
Điều hướng đến **màn hình cụ thể**

Khác với Navigator nội bộ:

Đây là **navigation từ hệ điều hành → app**

Mục tiêu học tập

Sinh viên có thể:

Cấu hình app Flutter mở bằng link (ví dụ: `myapp://profile/123`)
Hiểu khái niệm **deep linking**
Liên kết app với email, web, QR code

Mô tả sư phạm ngắn

Đây là bước chuyển:

từ **app đóng → app kết nối hệ sinh thái**

Rất phổ biến trong app thương mại, ngân hàng, mạng xã hội

Học để làm gì trong app thật?

→ Click link trong email mở đúng màn hình
→ Quét QR mở app
→ Chuyển từ web → app mobile

TỔNG MỤC TIÊU SAU SLOT 9

Sau Slot 9, sinh viên:

Không còn sợ layout phức tạp
Biết **chọn đúng widget cho đúng bài toán UI**
Hiểu navigation không chỉ là "push / pop"
Sẵn sàng học:

Responsive nâng cao
Navigation 2.0
UI/UX thực tế & animation

TÀI LIỆU THAM KHẢO UY TÍN

1 Flutter Official – Layout

PhucPT10@fpt.edu.vn

https://docs.flutter.dev/ui/layout

👉 Nguồn gốc chuẩn, xem để hiểu bản chất

2 **Flutter Official – Row & Column**

https://docs.flutter.dev/ui/layout#flex

👉 Rất quan trọng cho sinh viên mới

3 **Flutter GridView**

https://api.flutter.dev/flutter/widgets/GridView-class.html

👉 Dùng khi làm app dữ liệu thật

4 **Stack & Positioned**

https://api.flutter.dev/flutter/widgets/Stack-class.html

👉 Hiểu để tránh layout "hack"

5 **Deep Linking / URL Scheme**

https://docs.flutter.dev/ui/navigation/deep-linking

👉 Kết nối app với hệ điều hành

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1 iOS URL Scheme Navigation (External Navigation)

Khái niệm quan trọng (BẮT BUỘC HIỂU)

URL Scheme là cơ chế để:

Hệ điều hành iOS mở app Flutter

Điều hướng tới **một màn hình cụ thể** trong app

Phân biệt rõ:

Internal Navigation → `Navigator.push()`

External Navigation → app được mở từ **bên ngoài** (link, QR, email)

Tư duy cốt lõi: *Navigation không chỉ xảy ra bên trong app.*

Kỹ thuật / thao tác PHẢI THÀNH THẠO

Khai báo URL Scheme cho app iOS

Nhận và xử lý link mở app

Điều hướng đến đúng màn hình dựa trên dữ liệu từ URL

Phân tích luồng: **OS → App → Screen**

Lỗi thường gặp của sinh viên mới

Nghĩ rằng URL Scheme "chỉ là link mở app"

Không hiểu dữ liệu từ link đi vào đâu

Nhầm lẫn giữa:

route nội bộ

deep link từ hệ điều hành

Nguyên nhân gốc rễ:

Tư duy app là "đóng kín", không thấy vai trò của hệ điều hành.

Best Practices (chuẩn ngay từ đầu)

Luôn vẽ **luồng điều hướng** trước khi code

Tách:

xử lý link

xử lý điều hướng UI

Chỉ dùng URL Scheme cho **external entry point**, không thay thế Navigator nội bộ

Mô tả sư phạm

Nếu không hiểu URL Scheme, sinh viên sẽ:

Không làm được app thương mại thật

Bị "choáng" khi app phải mở từ email, web, QR

→ Đây là bước chuyển từ **app học tập → app sản phẩm**

2 Flex Layout – `Row` & `Column` (Tư duy bố cục theo trục)

Khái niệm quan trọng (BẮT BUỘC HIỂU)

`Row` / `Column` là **Flex layout**:

sắp xếp widget theo **1 trục chính**

Hiểu đúng:

`mainAxis` = trục sắp xếp

`crossAxis` = trục vuông góc

`Expanded` ≠ "cho đầy màn hình"

→ mà là **chia không gian linh hoạt**

Kỹ thuật / thao tác PHẢI THÀNH THẠO

Dựng UI bằng Row / Column có thứ tự hợp lý

Sử dụng `MainAxisAlignment` & `CrossAxisAlignment`

Dùng `Expanded`, `Spacer` đúng mục đích

Tránh overflow khi màn hình nhỏ

Lỗi thường gặp của sinh viên mới

Dùng Row/Column theo cảm tính

PhucPT10@fpt.edu.vn

Không hiểu vì sao UI bị tràn
Lạm dụng `SizedBox`, `Padding` để "chữa cháy"

👉 **Nguyên nhân gốc rễ**:
Không hiểu **trục layout**, chỉ nhìn bằng mắt chứ không tư duy không gian.

✅ Best Practices
Luôn hỏi: *UI này sắp xếp theo trục nào?*
Bố cục từ ngoài → vào trong
Ưu tiên Flex trước khi dùng các layout phức tạp hơn

📘 Mô tả sư phạm
Row & Column là **xương sống của Flutter UI**.
Không nắm vững Flex → Grid, Stack, Responsive đều sẽ sai từ gốc.

3 GridView – Layout dạng lưới (UI nhiều phần tử)

◈ Khái niệm quan trọng (BẮT BUỘC HIỂU)

`GridView` dùng khi UI:
- không còn tuyến tính
- cần hiển thị **nhiều item đồng dạng**

Grid = **ListView mở rộng theo 2 chiều**

🛠 Kỹ thuật / thao tác PHẢI THÀNH THẠO

Dùng `GridView.count` cho dữ liệu đơn giản
Dùng `GridView.builder` cho dữ liệu động
Điều chỉnh số cột, tỉ lệ item
Kết hợp Grid + navigation sang màn hình chi tiết

⚠ Lỗi thường gặp

Dùng ListView để hiển thị UI dạng lưới
Nhét GridView trong Column gây lỗi layout
Không kiểm soát chiều cao item

👉 **Nguyên nhân gốc rễ**:
Không phân biệt **layout tuyến tính** vs **layout dạng khối**.

✅ Best Practices
Grid cho:
- sản phẩm
- hình ảnh
- danh mục

Không dùng Grid khi UI chỉ cần 1 cột
Luôn test với nhiều kích thước màn hình

📘 Mô tả sư phạm
GridView giúp sinh viên:
Bước sang UI giống app thương mại
Thoát khỏi tư duy "bài tập ListView"

4 Stack & Z-order UI (UI chồng lớp)

◈ Khái niệm quan trọng (BẮT BUỘC HIỂU)

`Stack` cho phép **xếp widget theo lớp**
Thứ tự widget = **thứ tự hiển thị**
`Positioned` giúp kiểm soát vị trí tương đối

🛠 Kỹ thuật / thao tác PHẢI THÀNH THẠO

Chồng icon / badge lên UI
Tạo card có nhãn nổi
Kết hợp Stack với Row / Column
Hiểu vì sao widget bị che / đè

PhucPT10@fpt.edu.vn

⚠ Lỗi thường gặp

Dùng Stack để thay thế Row / Column
Position cứng → UI vỡ trên màn hình khác
Không hiểu z-order

👉 **Nguyên nhân gốc rễ**:

Không phân biệt **layout theo trục** và **layout theo lớp**.

✅ Best Practices

Chỉ dùng Stack khi **thực sự cần chồng lớp**
Hạn chế position cứng
Stack để tạo điểm nhấn, không phải layout chính

📝 Mô tả sư phạm

Nếu không hiểu Stack, sinh viên:

Không làm được UI hiện đại
Rất dễ "layout hack" và UI khó maintain

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

1 ỨNG DỤNG SLOT 9 TRONG PHÁT TRIỂN DỰ ÁN FLUTTER THỰC TẾ

1. Xây dựng UI có cấu trúc rõ ràng (UI Architecture)

Trong app Flutter thực tế:

Row / Column dùng để:

chia **khu vực chức năng** (header – content – footer)

tổ chức **thứ tự thị giác**

Flex layout là lớp nền:

mọi Grid, Stack đều đặt **trên Flex**

👉 Trong dự án thật:

Mỗi màn hình luôn bắt đầu bằng:

Column (vertical flow)

Bên trong mới tách Row / Grid / Stack

⚠️ Nếu không nắm Flex:

UI nhìn "có vẻ đúng"

nhưng **sửa 1 chỗ vỡ 3 chỗ**

code rất khó maintain

2. Xử lý layout phức tạp (Danh sách – Lưới – Chồng lớp)

Trong app thật:

GridView xuất hiện rất nhiều:

sản phẩm

danh mục

gallery

Stack dùng để:

badge

icon nổi

overlay trạng thái

👉 Vai trò Slot 9:

Giúp sinh viên hiểu:

layout theo trục ≠ layout theo lớp

Không còn:

nhét Stack để "chữa cháy"

lạm dụng Padding / SizedBox

3. Điều hướng & mở app từ bên ngoài (Navigation thật sự)

Trong app thương mại:

App **không chỉ mở từ icon**

Mà mở từ:

link

email

QR

web → app

👉 **iOS URL Scheme** là bước đầu:

giúp sinh viên:

hiểu deep navigation

không sốc khi học Navigation 2.0 sau này

2 CÁC TÌNH HUỐNG THỰC TẾ (REAL-WORLD SCENARIOS)

PhucPT10@fpt.edu.vn

◈ TÌNH HUỐNG 1: App bán hàng nhỏ (Mini E-commerce)

Bối cảnh

App Flutter bán sản phẩm đơn giản
Có:
- trang danh sách
- trang chi tiết sản phẩm

Vấn đề gặp phải

Danh sách sản phẩm hiển thị:
- lệch
- không cân đối

UI vỡ trên màn hình nhỏ
Code rất khó chỉnh layout

Áp dụng Slot 9

Column:
- bố cục tổng màn hình

GridView.builder:
- hiển thị sản phẩm dạng lưới

Expanded:
- chia chiều cao hợp lý

Stack:
- gắn badge "SALE" lên ảnh

Mục tiêu đạt được

UI:
- gọn
- rõ
- giống app thương mại

Code:
- dễ mở rộng
- dễ thêm tính năng

◈ TÌNH HUỐNG 2: App giới thiệu công ty / Startup MVP

Bối cảnh

MVP Flutter:
- landing page
- danh sách dịch vụ

Yêu cầu:
- UI đẹp
- triển khai nhanh

Vấn đề

Sinh viên:
- xếp UI theo cảm tính
- dùng Column lồng Column

UI:
- khó chỉnh
- khó responsive

Áp dụng Slot 9

Row / Column:
- chia section rõ ràng

mainAxisAlignment / crossAxisAlignment:
- căn chỉnh chuẩn ngay từ đầu

GridView.count:
- hiển thị dịch vụ dạng lưới

PhucPT10@fpt.edu.vn

Mục tiêu đạt được

Tốc độ làm MVP nhanh hơn
Ít phải sửa layout
Dễ demo cho khách hàng

TÌNH HUỐNG 3: App sinh viên + mở app từ link (iOS)

Bối cảnh

App quản lý sinh viên / bài tập
Có email gửi link:
- xem thông tin
- xem thông báo

Vấn đề

Sinh viên:
- chỉ biết Navigator.push
- không hiểu mở app từ link

Áp dụng Slot 9

iOS URL Scheme:
- mở app từ email

Navigation nội bộ:
- điều hướng đến màn hình cụ thể

Phân biệt rõ:
- external navigation
- internal navigation

Mục tiêu đạt được

App:
- giống sản phẩm thật
- không còn "app đóng"

Sinh viên:
- hiểu tư duy hệ thống

3 CASE STUDY – ÁP DỤNG SLOT 9 THÀNH CÔNG

Bối cảnh dự án

MVP Flutter cho startup nhỏ
App:
- danh sách sản phẩm
- chi tiết
- mở từ link quảng cáo

Vấn đề ban đầu

UI:
- rối
- layout vỡ

Code:
- Column lồng sâu
- Stack dùng sai

Navigation:
- chỉ push/pop

Kỹ thuật Slot 9 áp dụng

Flex layout:
- dựng lại toàn bộ cấu trúc UI

GridView.builder:
- danh sách sản phẩm

Stack:

PhucPT10@fpt.edu.vn

badge / icon

URL Scheme:

mở app từ link quảng cáo

✅ Kết quả

UI:

ổn định

dễ responsive

Code:

dễ đọc

dễ sửa

MVP:

demo nhanh

dễ thuyết phục khách hàng

4 BÀI HỌC RÚT RA (LESSONS LEARNED)

✅ Điều QUAN TRỌNG NHẤT khi xây UI Flutter

Tư duy layout trước – code sau

Hiểu rõ:

trục

không gian

thứ tự hiển thị

⚠ Sai lầm sinh viên hay mắc

Xếp UI theo cảm tính

Lạm dụng Column / Row

Dùng Stack để "vá lỗi"

Không phân biệt:

layout

navigation

🧠 Nguyên tắc duy trì lâu dài

UI = cấu trúc, không phải trang trí

Ưu tiên:

dễ đọc

dễ maintain

Vẽ layout (hoặc phác) trước khi code

Luôn nghĩ:

"Nếu sửa UI này sau 1 tháng, mình có hiểu lại được không?"

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

🧠 PHẦN A. TƯ DUY LAYOUT TRONG FLUTTER

I. TƯ DUY LAYOUT DẠNG FLEX (FOUNDATION)

◈ Layout trong Flutter là gì?

Layout là quá trình Flutter:

đo kích thước widget
quyết định vị trí hiển thị trên màn hình

Flutter **không vẽ UI theo tọa độ tuyệt đối**, mà theo:

"cha quyết định không gian – con tự vẽ trong không gian đó"

👉 Đây gọi là **constraint-based layout**

◈ Vì sao Flutter KHÔNG dùng layout kiểu HTML/CSS?

HTML/CSS:

phụ thuộc trình duyệt
DOM phức tạp

Flutter:

render trực tiếp bằng engine
cần **đồng nhất trên iOS / Android**

👉 Vì vậy Flutter chọn:

layout **đơn giản, dự đoán được**
ít magic, ít phụ thuộc môi trường

◈ Khái niệm Flex-based Layout

Flex layout = sắp xếp widget theo **1 trục**

Flutter dùng Flex làm nền:

`Row`
`Column`

👉 Tư duy cốt lõi:

UI không phải là "xếp chồng thẻ",
mà là **phân chia không gian theo trục**

◈ Trục chính & trục phụ

Main Axis:

trục sắp xếp widget

Cross Axis:

trục vuông góc

Ví dụ:

`Row`:

mainAxis → ngang
crossAxis → dọc

`Column`:

mainAxis → dọc
crossAxis → ngang

⚠ Sinh viên thường hiểu sai

Nghĩ Row = "để ngang", Column = "để dọc"

Không ý thức:

trục
không gian
giới hạn màn hình

PhucPT10@fpt.edu.vn

Liên hệ học thuật

CSS Flexbox:

- `flex-direction`
- `justify-content`
- `align-items`

Android:

- `LinearLayout`

👉 Flutter **gần Flexbox hơn CSS truyền thống**

PHẦN B. FLEX LAYOUT – ROW & COLUMN

II. ROW & COLUMN – BẢN CHẤT & CÁCH HOẠT ĐỘNG

Row là gì? Column là gì?

`Row`:

- sắp xếp widget **theo chiều ngang**

`Column`:

- sắp xếp widget **theo chiều dọc**

👉 Quan trọng:

Chúng **không tự co giãn**

Chúng **tuân theo constraint của widget cha**

Khi nào dùng Row, khi nào dùng Column?

Dùng `Column` khi:

- UI có flow từ trên xuống

Dùng `Row` khi:

- các thành phần nằm cùng hàng

👉 Trong app thật:

Mỗi màn hình **luôn bắt đầu bằng Column**

Quan hệ mainAxis & crossAxis

`mainAxisAlignment`:

- căn chỉnh trên trục sắp xếp

`crossAxisAlignment`:

- căn chỉnh trên trục vuông góc

Ví dụ:

Header:

- Row
- mainAxis: spaceBetween

Card:

- Column
- crossAxis: start

❗ Vì sao UI hay bị lệch / overflow?

Không hiểu:

- trục
- giới hạn màn hình

Đặt quá nhiều widget trên mainAxis

Không chia không gian hợp lý

👉 **Lỗi tư duy**, không phải lỗi cú pháp

PhucPT10@fpt.edu.vn

🧠 PHẦN B (TIẾP). EXPANDED & FLEXIBLE

III. EXPANDED & FLEXIBLE – KIỂM SOÁT KHÔNG GIAN

◈ Expanded là gì?

`Expanded`:

chiếm hết không gian còn lại

bắt buộc trong Flex (Row / Column)

◈ Flexible khác Expanded thế nào?

`Flexible`:

được phép co giãn

không ép chiếm toàn bộ

👉 So sánh:

Expanded = "bắt buộc lấp đầy"

Flexible = "linh hoạt"

◈ Khi nào BẮT BUỘC dùng Expanded?

Row có nhiều widget:

text dài

button

Khi gặp lỗi:

`RenderFlex overflow`

⚠ Lỗi phổ biến

Không dùng Expanded → UI tràn

Dùng Expanded bừa → layout cứng

👉 Phải hiểu:

Expanded không sửa UI,

nó **phân chia không gian**

🧠 PHẦN C. GRIDVIEW – HIỂN THỊ DỮ LIỆU DẠNG LƯỚI

IV. GRIDVIEW – TƯ DUY & ỨNG DỤNG

◈ Vì sao cần GridView?

UI thực tế:

không chỉ là list dọc

Grid = list **2 chiều**

◈ GridView.count vs GridView.builder

`count`:

số cột cố định

demo, dữ liệu nhỏ

`builder`:

dữ liệu động

hiệu năng tốt

✕ Vì sao không dùng Column + Row để dựng grid?

Code:

dài

khó maintain

Không responsive

Hiệu năng kém

PhucPT10@fpt.edu.vn

👉 GridView giải quyết:
layout
scroll
hiệu năng

🧠 PHẦN D. STACK – UI CHỒNG LỚP

V. STACK & POSITIONED – Z-ORDER TRONG UI

◈ Stack là gì?

Stack:
xếp widget **theo lớp**
Widget sau:
nằm **trên** widget trước

◈ Khi nào cần chồng UI?

Badge
Icon nổi
Overlay text

◈ Positioned & alignment

`Positioned`:
đặt widget theo cạnh
`alignment`:
căn chỉnh tương đối

⚠ Cảnh báo

Stack không phải layout chính
Lạm dụng Stack → UI khó maintain

🧠 PHẦN E. iOS URL SCHEME NAVIGATION

VI. iOS URL SCHEME – ĐIỀU HƯỚNG TỪ HỆ ĐIỀU HÀNH

◈ URL Scheme là gì?

Cơ chế:
mở app từ **bên ngoài**
Ví dụ:
email
QR
link web

◈ Khác gì navigation nội bộ?

Navigator:
trong app
URL Scheme:
OS → app

👉 Đây là **external navigation**

◈ Ý nghĩa học thuật

App không còn là "hộp kín"
Chuẩn bị tư duy cho:
Deep Linking

PhucPT10@fpt.edu.vn

Navigation 2.0

🧠 PHẦN F. LỖI TƯ DUY & LỖI LAYOUT PHỔ BIẾN

VII. LỖI THƯỜNG GẶP TRONG SLOT 9

❌ Lỗi 1: Dùng sai Row / Column

Nguyên nhân:

không hiểu trục

Cách tránh:

xác định flow UI trước

❌ Lỗi 2: UI vỡ khi đổi màn hình

Nguyên nhân:

kích thước cứng

Cách sửa:

dùng Expanded, GridView

❌ Lỗi 3: Lạm dụng Stack

Nguyên nhân:

muốn sửa nhanh

Cách tránh:

Stack chỉ dùng cho overlay

❌ Lỗi 4: Nhầm layout & navigation

Nguyên nhân:

chưa hiểu luồng app

Cách sửa:

vẽ flow trước khi code

🧠 PHẦN G. TỔNG KẾT & ĐỊNH HƯỚNG

VIII. TỔNG KẾT SLOT 9

🎯 Slot 9 giúp sinh viên:

Thoát khỏi UI "xếp đại"

Hiểu layout là **bài toán tư duy**

Code UI đúng ngay từ đầu

🚀 Chuẩn bị cho slot tiếp theo:

Responsive Layout

Advanced Widgets

Navigation nâng cao

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1. Kỹ thuật: Row & Column (Flex Layout Core)

Kinh nghiệm thực tế trong dự án Flutter

Row / Column xuất hiện ở 100% màn hình app thật, ví dụ:

Header (avatar – tên – action)
Card thông tin
Profile
Action bar

Bối cảnh dự án

App thương mại nhỏ, màn hình sản phẩm

Vấn đề

UI nhìn đúng trên máy dev
Nhưng sang máy khác:
text tràn
nút bị đẩy ra ngoài

Vì sao chọn Row / Column

UI có flow rõ ràng theo trục
Không cần grid, không cần stack

Lỗi thường gặp

Lỗi 1: Dùng Row / Column theo cảm tính

Biểu hiện:
UI lệch
spacing không đều

Nguyên nhân gốc:
Không xác định *trục chính*

Ví dụ:
Dùng Row nhưng lại căn chỉnh như Column

Cách khắc phục

Luôn trả lời trước khi code:
"UI này chảy theo trục nào?"

Phòng tránh

Vẽ layout (hoặc tưởng tượng) trước khi viết widget

Mẹo lập trình

Debug layout bằng:

```
Container(color: Colors.red.withOpacity(0.1))
```

Luôn đọc code theo thứ tự widget tree

Best Practices

Mỗi màn hình:
bắt đầu bằng Column
Không lồng Column quá sâu (2–3 cấp là giới hạn an toàn)

Bài học cho sinh viên

Nếu không hiểu Row / Column,
mọi layout phức tạp sau này đều "đoán mò".

2. Kỹ thuật: Expanded vs Flexible

Kinh nghiệm thực tế

Expanded thường dùng ở:

PhucPT10@fpt.edu.vn

Row có text dài
Header có nhiều thành phần
Layout co giãn theo màn hình

📍 **Bối cảnh**

App có title dài + button

❗ **Vấn đề**

`RenderFlex overflow` trên máy nhỏ

🎯 **Vì sao chọn Expanded**

Cần **chia không gian**, không phải set kích thước cứng

⚠ Lỗi thường gặp

✕ *Lỗi 1: Không dùng Expanded khi cần*

Biểu hiện: overflow vàng/đen

Nguyên nhân:

Text chiếm hết mainAxis

🛠 **Cách sửa**

Bọc widget "ăn gian không gian" bằng Expanded

✕ *Lỗi 2: Lạm dụng Expanded*

Biểu hiện:

UI cứng, khó kiểm soát

Nguyên nhân:

Dùng Expanded như "thuốc chữa bách bệnh"

🚫 **Phòng tránh**

Chỉ dùng Expanded khi:

Có *nhiều widget tranh chấp không gian*

💡 Mẹo

Nếu **Expanded làm layout xấu đi** → có thể bạn cần Flexible

🛡 Best Practices

Row:

text dài → Expanded
icon → kích thước tự nhiên

Tránh hard-code width trong Row

🎓 Bài học

Expanded không phải để "cho đầy",
mà để **chia không gian hợp lý**.

◈ 3. Kỹ thuật: GridView.count & GridView.builder

☑ Kinh nghiệm thực tế

GridView dùng trong:

Danh sách sản phẩm
Danh mục
Gallery ảnh

📍 **Bối cảnh**

App bán hàng, 20–50 sản phẩm

❗ **Vấn đề**

Dùng Column + Row:

code dài

PhucPT10@fpt.edu.vn

không scroll mượt
không responsive

🎯 **Vì sao chọn GridView**

Có sẵn:

scroll
layout
tối ưu hiệu năng

⚠ Lỗi thường gặp

❌ *Lỗi 1: Tự dựng grid bằng Row/Column*

Biểu hiện:

code khó đọc
lỗi layout khi thêm item

Nguyên nhân:

Chưa hiểu GridView là layout chuyên dụng

🛠 **Cách sửa**

Chuyển sang GridView.builder

❌ *Lỗi 2: Dùng GridView.count cho dữ liệu động*

Nguyên nhân:

Không hiểu sự khác nhau với builder

Hậu quả:

hiệu năng kém khi dữ liệu lớn

💡 Mẹo

Data từ API → **luôn dùng GridView.builder**

Test grid với:

item ít
item rất nhiều

🛡 Best Practices

Không nhét GridView trong Column nếu không có Expanded

GridView = layout + scroll, đừng "bọc thêm scroll"

🎓 Bài học

Khi UI có dạng lưới,
GridView là bắt buộc, không phải lựa chọn.

4. Kỹ thuật: Stack & Positioned

✅ Kinh nghiệm thực tế

Stack dùng cho:

Badge thông báo
Icon nổi trên ảnh
Overlay text

📍 **Bối cảnh**

Card sản phẩm có nhãn "SALE"

❗ **Vấn đề**

Badge không cố định
UI lệch khi đổi kích thước

🎯 **Vì sao chọn Stack**

Cần **chồng lớp**, không phải sắp xếp theo trục

PhucPT10@fpt.edu.vn

⚠ Lỗi thường gặp

✕ *Lỗi 1: Dùng Stack làm layout chính*

Biểu hiện:

code khó đọc
khó maintain

Nguyên nhân:

Nhầm Stack với Row/Column

🛠 **Cách sửa**

Stack chỉ làm **overlay**, không làm khung chính

✕ *Lỗi 2: Positioned cứng*

Biểu hiện:

UI đẹp trên 1 máy, vỡ trên máy khác

Nguyên nhân:

hard-code top/left

🚫 **Phòng tránh**

Ưu tiên alignment
Tránh position tuyệt đối nếu không cần

💡 Mẹo

Nhìn Stack như Photoshop:

mỗi widget = 1 layer

🛡 Best Practices

Stack:

dùng ít
dùng đúng

Nếu không cần chồng → **không dùng Stack**

🎓 Bài học

Stack mạnh, nhưng dùng sai sẽ phá toàn bộ layout.

◈ 5. Kỹ thuật: iOS URL Scheme Navigation

☑ Kinh nghiệm thực tế

Dùng khi:

Mở app từ email
QR code
Link quảng cáo

📍 **Bối cảnh**

App có link: "Xem chi tiết đơn hàng"

❗ **Vấn đề**

Sinh viên chỉ biết Navigator.push
Không hiểu app được mở từ đâu

🎯 **Vì sao dùng URL Scheme**

Đây là **external navigation**
App thật **không phải lúc nào cũng mở từ icon**

⚠ Lỗi thường gặp

✕ *Lỗi 1: Nhầm URL Scheme với route nội bộ*

Biểu hiện:

PhucPT10@fpt.edu.vn

không vào đúng màn hình

Nguyên nhân:

không hiểu OS → app → screen

🛠 **Cách sửa**

Tách:

xử lý link

xử lý navigation

💡 Mẹo

Vẽ luồng:

Link → App → Screen

trước khi code

🛡 Best Practices

URL Scheme:

chỉ dùng cho entry point từ bên ngoài

Không thay thế Navigator nội bộ

🎓 Bài học

App thật luôn sống trong **hệ sinh thái**,
không bao giờ là "hộp kín".

🎯 TỔNG KẾT THỰC CHIẾN SLOT 9

Điều quan trọng nhất sinh viên cần nhớ:

UI không crash ≠ UI đúng

Layout là **tư duy**, không phải widget

Navigation gắn với **hệ điều hành**, không chỉ trong app

👉 Nếu nắm vững phần này, sinh viên **sẵn sàng bước sang**:

Responsive UI

Advanced Layout

Navigation 2.0 & Deep Linking

PhucPT10@fpt.edu.vn

EXERCISE

📄 BÀI TẬP TỔNG HỢP 1

(Navigation – Named Routes – Navigator 1.0/2. 0 – Deep Linking)

1 TỔNG QUAN DỰ ÁN

◈ Tên dự án

`Simple Product Dashboard App`

◈ Mô tả

Ứng dụng Flutter mô phỏng dashboard sản phẩm đơn giản gồm:

- **Trang Dashboard** (Flex Layout: `Column`, `Row`, `Expanded`, `Stack`)
- **Trang Danh sách sản phẩm** (dùng `GridView.builder`)
- **UI badge / overlay** (dùng `Stack` + `Positioned`)
- *Nhận thức* về **Mở app & điều hướng từ iOS URL Scheme** (deep link)

👉 Đây là app **demo layout & navigation**, *không* phải app bán hàng hoàn chỉnh.

◈ Mục tiêu học tập

Sau dự án này, sinh viên sẽ:

- Hiểu tư duy **layout theo trục** (trục dọc/trục ngang trong `Column`/`Row`)
- Biết khi nào dùng:
  - `Row` / `Column` / `Expanded`
  - Khi nào phải chuyển sang `GridView` (khi có nhiều item lặp lại, dạng lưới)
- Hiểu `Stack` dùng để **chồng UI (overlay)** – *không* phải xếp layout chính
- Nhận thức được:
  "Navigation không chỉ là bấm nút trong app – mà còn đến từ hệ điều hành (OS) qua URL, notification, deep link, ..."

◈ Đối tượng phù hợp

- Đã học Flutter cơ bản
- Viết được `StatelessWidget` / `StatefulWidget`
- Hay bị lỗi layout, overflow, UI lệch
- Chưa tự tin làm UI "giống app thật", dễ đọc, dễ sửa

2 CHỨC NĂNG CHÍNH CỦA ỨNG DỤNG

◈ Màn hình 1: `DashboardScreen`

Dùng:

- `Column` → layout dọc
- `Row` → header
- `Expanded` → chia không gian linh hoạt
- `Stack` → badge thông báo (chấm đỏ nhỏ đặt chồng lên icon chuông)

👉 **Mục đích**:

- Rèn tư duy **Flex Layout**
- Hiểu rõ:
  - `mainAxis` (trục chính)
  - `crossAxis` (trục phụ)

PhucPT10@fpt.edu.vn

◈ Màn hình 2: `ProductGridScreen`
Dùng:
- `GridView.builder`
- `Card` sản phẩm (UI đơn giản)
- Có **scroll** mượt

👉 **Mục đích**:
- Hiểu vì sao `Column` + `Row` **không thay thế được** `GridView`
- Biết dùng `GridView.builder` cho danh sách nhiều phần tử

◈ Navigation
- Điều hướng **trong app**: `Navigator.push`
- **Nhận thức** về điều hướng **từ ngoài app**: iOS URL Scheme

Ví dụ deep link:

```Text
myapp://products
```

3 CÔNG NGHỆ & CÔNG CỤ
- Flutter SDK
- Dart
- IDE: VS Code / Android Studio
- Simulator: iOS Simulator / Android Emulator đều được
  (deep link iOS: chỉ *giải thích*, không bắt buộc làm đủ cho bài này)
- Material Widgets

👉 **Không dùng** Provider / Bloc / Clean Architecture.
Chỉ tập trung: **UI + Layout + Cơ bản Navigation.**

4 CẤU TRÚC THƯ MỤC DỰ ÁN

```Text
lib/
 ├── main.dart
 ├── screens/
 │    ├── dashboard_screen.dart
 │    ├── product_grid_screen.dart
 ├── widgets/
 │    ├── product_card.dart
 │    ├── dashboard_header.dart
```

💡 **Vì sao phải tách widget?**
- File dài → **khó đọc**
- Khó debug lỗi `overflow`
- Không tái sử dụng được
- Dễ "rối mắt" với sinh viên mới

👉 Tách widget = **dễ sửa UI + dễ mở rộng + dễ debug**

5 CÁC BƯỚC THỰC HIỆN (TÓM TẮT)
- Tạo project Flutter
- Tạo cấu trúc thư mục
- Dựng `DashboardScreen` (Flex Layout)
- Fix lỗi overflow bằng `Expanded`

PhucPT10@fpt.edu.vn

Thêm `Stack` & badge thông báo
Tạo `ProductGridScreen`
Dùng `GridView.builder` + `ProductCard`
Điều hướng giữa các màn hình (Navigator.push, named routes)
Giải thích khái niệm iOS URL Scheme (deep link)
Test app:
    Không overflow
    Navigation OK
    GridView scroll được

6 HƯỚNG DẪN CHI TIẾT TỪNG BƯỚC

🌸 BƯỚC 1: Tạo project & cấu trúc thư mục

📌 *Mục tiêu*

Có project Flutter chạy được
Cấu trúc thư mục rõ ràng ngay từ đầu

🛠 *Thao tác*

Mở terminal (VS Code / Android Studio)
Chạy lệnh:

```bash
flutter create simple_dashboard_app
```

Mở thư mục dự án:

```bash
cd simple_dashboard_app
```

Tạo thư mục:

```
lib/screens
lib/widgets
```

✅ *Kiểm tra*

Chạy lệnh:

```bash
flutter run
```

Thấy app Flutter mặc định (màn hình counter) → **OK**
Lúc này chưa sửa gì trong `lib/main.dart` (vẫn code mặc định của Flutter).

🌸 BƯỚC 2: Sửa `main.dart` – Khai báo routes & mở Dashboard

📌 *Mục tiêu*

Hiểu file `main.dart` là **entry point**
Dùng **named routes** để chuyển trang

🛠 *Thao tác*

Mở file `lib/main.dart`
Xóa code mặc định, thay bằng code sau:

lib/main.dart

```
import 'package:flutter/material.dart';
import 'screens/dashboard_screen.dart';
import 'screens/product_grid_screen.dart';

void main() {
  runApp(const MyApp());
```


PhucPT10@fpt.edu.vn

```
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Simple Product Dashboard',
      debugShowCheckedModeBanner: false,
      // route mặc định khi app mở lên
      initialRoute: '/',
      // Định nghĩa named routes
      routes: {
        '/': (context) => const DashboardScreen(),
        '/products': (context) => const ProductGridScreen(),
      },
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
    );
  }
}
```

✅ *Kiểm tra*

Chạy `flutter run`

Nếu **chưa tạo** `DashboardScreen`, sẽ báo lỗi import → **bước sau sẽ tạo**, sau đó chạy lại.

🧩 BƯỚC 3: Tạo `DashboardScreen` (Flex Layout)

📌 *Mục tiêu*

Hiểu layout theo **trục dọc** (`Column`)

Biết dùng `Expanded` để tránh overflow

🛠️ *Thao tác*

Tạo file `lib/screens/dashboard_screen.dart`

Thêm code:

lib/screens/dashboard_screen.dart

```
import 'package:flutter/material.dart';
import '../widgets/dashboard_header.dart';

class DashboardScreen extends StatelessWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Dashboard')),
      body: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          // Phần header (row + stack)
          const DashboardHeader(),
```


PhucPT10@fpt.edu.vn

```
          // Khoảng cách nhỏ
          const SizedBox(height: 16),

          // Phần nội dung chính chiếm toàn bộ không gian còn lại
          Expanded(
            child: Center(
              child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  const Text(
                    'Welcome to Simple Product Dashboard',
                    style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold),
                    textAlign: TextAlign.center,
                  ),
                  const SizedBox(height: 16),
                  ElevatedButton(
                    onPressed: () {
                      // Điều hướng đến màn hình Grid qua named route
                      Navigator.pushNamed(context, '/products');
                    },
                    child: const Text('Xem danh sách sản phẩm'),
                  ),
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

Lưu ý: `DashboardHeader` chưa tồn tại, sẽ tạo ở **Bước 4**.

⚠ *Lỗi thường gặp*

- ✕ Không dùng `Expanded` cho phần nội dung → `RenderFlex overflowed…`
- ✕ Quên `crossAxisAlignment` → khó căn chỉnh UI ngang

💡 *Tip*

Trong `Column`, bất cứ phần nào **chiếm nhiều không gian** → hãy **nghĩ đến `Expanded`** trước.
`mainAxis` trong `Column` là **dọc**, trong `Row` là **ngang**.

✅ *Kiểm tra*

Tạm thời comment dòng `const DashboardHeader()`, nếu chưa tạo header.
Chạy app:

Thấy màn hình có text **"Welcome to Simple Product Dashboard"** và **button**.
Bấm button → nếu đã khai báo route `/products` nhưng chưa tạo `ProductGridScreen` sẽ bị lỗi → **chưa sao**, sẽ sửa ở bước sau.

🧩 BƯỚC 4: Tạo `DashboardHeader` dùng `Row` + `Stack`

📌 *Mục tiêu*

Hiểu `Row` để bố trí ngang

PhucPT10@fpt.edu.vn

Hiểu `Stack` để **chồng** badge lên icon

🛠 *Thao tác*

Tạo file `lib/widgets/dashboard_header.dart`

Thêm code:

lib/widgets/dashboard_header.dart

```
import 'package:flutter/material.dart';

class DashboardHeader extends StatelessWidget {
  const DashboardHeader({super.key});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(16),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          // Bên trái: lời chào
          Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: const [
              Text(
                'Hello, User 👋',
                style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
              ),
              SizedBox(height: 4),
              Text(
                'Welcome back to your dashboard',
                style: TextStyle(color: Colors.grey),
              ),
            ],
          ),
          // Bên phải: icon notification + badge
          Stack(
            clipBehavior: Clip.none,
            children: [
              const Icon(
                Icons.notifications,
                size: 28,
              ),
              Positioned(
                right: -2,
                top: -2,
                child: CircleAvatar(
                  radius: 6,
                  backgroundColor: Colors.red,
                ),
              ),
            ],
          ),
        ],
      ),
```


PhucPT10@fpt.edu.vn

```
    );
  }
}
```

⚠️ *Lỗi thường gặp*

- ❌ Dùng `Stack` để **xếp toàn bộ layout** (header + body + footer) → rất khó maintain, khó responsive
- ❌ Quên `clipBehavior: Clip.none` rồi dùng `Positioned` lệch ra ngoài → badge bị cắt mất

💡 *Tip*

`Stack` chỉ nên dùng cho:

Badge thông báo
Nổi hình avatar lên background
Overlay loading...

👉 **Không** dùng Stack để "xếp đại UI", hãy ưu tiên `Column/Row`.

✅ *Kiểm tra*

Mở lại `DashboardScreen`, bỏ comment `const DashboardHeader()`, nếu có.
Chạy app:

Thấy bên trái: dòng "Hello, User 👋 ..."
Bên phải: icon chuông có chấm đỏ nhỏ
Không có lỗi overflow.

🧩 BƯỚC 5: Tạo `ProductGridScreen` & `ProductCard`

📌 *Mục tiêu*

Hiểu `GridView.builder` để tạo danh sách dạng lưới
So sánh với `Column` + `Row`

🛠️ *Thao tác: Tạo `ProductCard`*

Tạo file `lib/widgets/product_card.dart`
Thêm code:

lib/widgets/product_card.dart

```
import 'package:flutter/material.dart';

class ProductCard extends StatelessWidget {
  final String title;
  final double price;

  const ProductCard({
    super.key,
    required this.title,
    required this.price,
  });

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 2,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(8),
      ),
      child: Padding(
```


PhucPT10@fpt.edu.vn

```
        padding: const EdgeInsets.all(8),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // Ảnh giả (placeholder)
            Expanded(
              child: Container(
                decoration: BoxDecoration(
                  color: Colors.blue[100],
                  borderRadius: BorderRadius.circular(8),
                ),
                child: const Center(
                  child: Icon(
                    Icons.shopping_bag,
                    size: 40,
                    color: Colors.blue,
                  ),
                ),
              ),
            ),
            const SizedBox(height: 8),
            Text(
              title,
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 4),
            Text(
              '\$${price.toStringAsFixed(2)}',
              style: const TextStyle(color: Colors.green),
            ),
          ],
        ),
      ),
    );
  }
}
```

Ở đây dùng `Expanded` bên trong `Column` để phần "ảnh" tự giãn chiếm hết chiều cao còn lại trong card.

🛠 *Thao tác: Tạo `ProductGridScreen`*

Tạo file `lib/screens/product_grid_screen.dart`

Thêm code:

lib/screens/product_grid_screen.dart

```
import 'package:flutter/material.dart';
import '../widgets/product_card.dart';

class ProductGridScreen extends StatelessWidget {
  const ProductGridScreen({super.key});

  @override
  Widget build(BuildContext context) {
```


PhucPT10@fpt.edu.vn

```
    // Dữ liệu giả cho 10 sản phẩm
    final products = List.generate(
      10,
      (index) => {
        'title': 'Product #$index',
        'price': 10.0 + index,
      },
    );

    return Scaffold(
      appBar: AppBar(
        title: const Text('Products'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: GridView.builder(
          itemCount: products.length,
          gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: 2, // 2 cột
            childAspectRatio: 0.75, // tỉ lệ ngang/dọc của mỗi item
            crossAxisSpacing: 12,
            mainAxisSpacing: 12,
          ),
          itemBuilder: (context, index) {
            final product = products[index];
            return ProductCard(
              title: product['title'] as String,
              price: product['price'] as double,
            );
          },
        ),
      ),
    );
  }
}
```

⚠ *Lỗi thường gặp*

- ✕ Dùng `Column` + nhiều `Row` để vẽ lưới → **không scroll được**, rất khó maintain, dễ vỡ UI
- ✕ Bọc `GridView` trong `Column` mà không có `Expanded` → lỗi `Vertical viewport was given unbounded height`
- ✕ Để `itemCount` quá lớn mà không dùng `builder` → tốn RAM

💡 *Tip*

Nếu hiển thị **danh sách lặp lại** và có thể dài → hãy nghĩ đến:

`ListView.builder` (danh sách một cột)

`GridView.builder` (danh sách dạng lưới)

✅ *Kiểm tra*

Mở app → Dashboard

Bấm nút **"Xem danh sách sản phẩm"**

Thấy màn hình mới có grid 2 cột, scroll được → **OK**

PhucPT10@fpt.edu.vn

🧩 BƯỚC 6: Điều hướng trong app (`Navigator.push` & Named Route)

📌 *Mục tiêu*

Hiểu sự khác nhau:

```
Navigator.push(context, MaterialPageRoute(...))
Navigator.pushNamed(context, '/routeName')
```

🛠 *Thao tác*

Đã làm trong `DashboardScreen`:

```dart
ElevatedButton(
  onPressed: () {
    Navigator.pushNamed(context, '/products');
  },
  child: const Text('Xem danh sách sản phẩm'),
),
```

Cách điều hướng **không dùng named route** (chỉ để so sánh):

```dart
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => const ProductGridScreen(),
  ),
);
```

👉 Ở bài này, ta ưu tiên **named route** để:

Rõ ràng

Dễ map với deep link về sau (`/products`)

✅ *Kiểm tra*

Bấm back trên `AppBar` ở `ProductGridScreen` → quay lại Dashboard

Bấm nút đi tới Products nhiều lần → Stack navigation hoạt động bình thường.

🧩 BƯỚC 7: iOS URL Scheme Navigation (NHẬN THỨC – KHÔNG YÊU CẦU CẤU HÌNH ĐỦ)

📌 *Mục tiêu*

Hiểu khái niệm:

Ứng dụng có thể được mở bởi **OS** thông qua **URL Scheme**

Khi app mở từ URL, có thể điều hướng đến màn hình tương ứng

Ví dụ: URL

```text
myapp://products
```

👉 Khi nhấn URL (từ Safari / app khác):

Hệ điều hành (iOS) mở app `myapp`

App nhận URL

App điều hướng tới màn hình `/products`

🧠 *Mức độ yêu cầu trong bài này*

Không bắt buộc sinh viên cấu hình thật trong iOS (`Info.plist`, `SceneDelegate`…)

Chỉ **hiểu concept**:

"Link ≠ button"

Navigation có thể đến từ **ngoài app**, chứ không chỉ `Navigator.push` trong code

Với sinh viên khá, có thể phát triển thêm:

Đọc thêm về `uni_links` package

PhucPT10@fpt.edu.vn

Cấu hình iOS URL Scheme
Trong `main.dart`, xử lý URL nhận được để `Navigator.pushNamed(context, '/products')`.

7 KIỂM TRA & CHẠY ỨNG DỤNG

◈ Test Dashboard

Mở app:

- Thấy `AppBar` title: **Dashboard**
- Thấy header với "Hello, User 👋" và icon chuông có badge đỏ
- Không có lỗi overflow (không có dải vàng/đen ở log)

Thử:

- Rotate màn hình (trên simulator) để xem có overflow không

◈ Test Product Grid

Từ Dashboard → bấm nút **Xem danh sách sản phẩm**

Trên `ProductGridScreen`:

- Thấy grid 2 cột
- Scroll xuống mượt
- Card hiển thị:
  - Icon túi hàng
  - Tên sản phẩm: `Product #0`, `Product #1`, ...
  - Giá: `$10.00`, `$11.00`, ...

◈ Test (nhận thức) iOS URL Scheme

Giải thích cho sinh viên:

- Nếu cấu hình `myapp://products`:
  - Khi người dùng mở link này, app sẽ nhận được thông tin `"/products"`
  - App có thể gọi:

```dart
Navigator.pushNamed(context, '/products');
```

Không yêu cầu các em thực thi đủ phần này trong bài (vì khá dài và liên quan đến native iOS).

8 CÁC LỖI PHỔ BIẾN & CÁCH XỬ LÝ

Lỗi	Nguyên nhân	Cách sửa
Flex overflow	Quên `Expanded` trong `Column/Row`	Bọc widget chiếm không gian bằng `Expanded`
`Vertical viewport...`	Bọc `GridView` trong `Column` không `Expanded`	Dùng `Expanded(child: GridView(...))` hoặc dùng `Scaffold` trực tiếp
Grid không scroll	Dùng `Column/Row` xếp item	Dùng `GridView.builder`
Stack che nội dung	`Positioned` sai, không set `clipBehavior`	Điều chỉnh `right/top`, set `clipBehavior: Clip.none` nếu cần

PhucPT10@fpt.edu.vn

Tên route sai	Gõ nhầm `'/product'` vs `'/products'`	Kiểm tra lại key trong `routes` và tên route dùng khi `push`
Import lỗi	Sai đường dẫn `../widgets/...`	Kiểm tra lại cấu trúc thư mục và câu lệnh `import`

9 KẾT QUẢ ĐẠT ĐƯỢC

Sau khi hoàn thành bài tập, sinh viên:

- ✅ Không còn xếp UI "theo cảm giác"
- ✅ Biết chọn layout đúng cho từng vấn đề:
  `Column` / `Row` / `Expanded` / `Stack` / `GridView`
- ✅ Hiểu navigation không chỉ là `Navigator.push` trong app
  Mà còn đến từ **hệ điều hành** (deep link, notification, URL, ...)
- ✅ Sẵn sàng học tiếp:
  Responsive layout (MediaQuery, LayoutBuilder)
  Navigation nâng cao (Navigator 2.0, go_router, deep link thật)

PhucPT10@fpt.edu.vn

TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

Nhóm	Thuật ngữ	Định nghĩa ngắn gọn (dễ hiểu)	Dùng khi nào trong app thật?	Lỗi sinh viên hay gặp
Flutter UI	Widget	Đơn vị UI nhỏ nhất trong Flutter	Mọi thành phần UI	Nghĩ widget chỉ là hiển thị
	Widget Tree	Cấu trúc cha–con của widget	Debug layout, tổ chức UI	Đặt widget sai vị trí
	Scaffold	Khung màn hình chuẩn	Mỗi screen chính	Lồng Scaffold
Flex Layout	Flex Layout	Layout theo trục (1 chiều)	Header, form, section	Xếp UI theo cảm tính
	Row	Sắp xếp con theo ngang	Icon + Text, Action bar	Dùng Row cho layout dọc
	Column	Sắp xếp con theo dọc	Màn hình, form	Lạm dụng Column
	mainAxis	Trục chính của Row/Column	Căn layout đúng trục	Nhầm trục
	crossAxis	Trục vuông góc	Căn chỉnh phụ	Căn sai hướng
	mainAxisAlignment	Canh theo trục chính	Chia khoảng cách	Căn không như mong muốn
	crossAxisAlignment	Canh theo trục phụ	Canh giữa/đầu/cuối	UI lệch
Space Control	Expanded	Chiếm hết không gian còn lại	Fix overflow	Quên dùng
	Flexible	Co giãn linh hoạt	UI linh hoạt	Nhầm với Expanded
	RenderFlex Overflow	Lỗi tràn layout	Khi UI vỡ	Không hiểu nguyên nhân
GridView	GridView	Hiển thị dạng lưới	Sản phẩm, ảnh	Dùng Column thay Grid
	GridView.count	Grid số cột cố định	UI đơn giản	Không responsive
	GridView.builder	Grid động, hiệu năng tốt	Danh sách lớn	Không dùng builder
	SliverGridDelegate	Cách chia ô lưới	Điều chỉnh grid	Bỏ qua cấu hình
Stack	Stack	Chồng widget lên nhau	Badge, overlay	Lạm dụng Stack
	Z-order	Thứ tự trước–sau	Layer UI	Không hiểu thứ tự
	Positioned	Định vị trong Stack	Icon, badge	Che nội dung
Navigation	Navigation	Điều hướng trong app	Chuyển màn hình	Nhầm layout vs navigation
	Route	Địa chỉ màn hình	Push / Pop	Route rối
	iOS URL Scheme	Mở app từ link ngoài	Email, QR	Cấu hình sai
	Deep Linking	Mở app tới màn cụ thể	Marketing, share link	Nhầm với navigation nội bộ
State	StatelessWidget	UI không đổi	Layout tĩnh	Dùng sai chỗ
	StatefulWidget	UI thay đổi	UI động	Quên setState
	setState	Cập nhật UI	Khi data đổi	Gọi sai chỗ

PhucPT10@fpt.edu.vn

I. THUẬT NGỮ CHUNG VỀ FLUTTER & UI

◈ Widget

Định nghĩa:
Đơn vị UI nhỏ nhất trong Flutter. Mọi thứ đều là Widget (text, button, layout, screen).
Hiểu đơn giản:
👉 Widget = **viên gạch xây UI**
Trong app thật:
Mỗi màn hình = nhiều widget lồng nhau
Sinh viên hay hiểu sai:
❌ Widget chỉ là "cái hiển thị"
✅ Widget vừa **hiển thị** vừa **quản lý layout**

◈ Widget Tree

Định nghĩa:
Cấu trúc cây thể hiện quan hệ cha – con giữa các widget.
Hiểu đơn giản:
👉 UI Flutter = **cây gia phả widget**
Trong app thật:
Lỗi layout thường do **đặt widget sai vị trí trong cây**

◈ Scaffold

Định nghĩa:
Widget cung cấp **khung giao diện chuẩn** cho một màn hình.
Bao gồm:
AppBar
Body
FloatingActionButton
Hiểu đơn giản:
👉 Scaffold = **bộ khung căn nhà**
Sai lầm phổ biến:
❌ Dùng Scaffold lồng Scaffold

II. THUẬT NGỮ VỀ FLEX LAYOUT (CỐT LÕI SLOT 9)

◈ Flex Layout

Định nghĩa:
Kiểu layout sắp xếp widget **theo trục** (1 chiều).
Hiểu đơn giản:
👉 Sắp xếp **theo hàng hoặc theo cột**
Liên hệ:
CSS Flexbox
Android LinearLayout

◈ Row

Định nghĩa:
Widget sắp xếp con **theo chiều ngang**
Trục chính (mainAxis):
➡️ Trái → Phải
Dùng khi:

PhucPT10@fpt.edu.vn

Header
Action bar
Icon + Text

◈ Column

Định nghĩa:
Widget sắp xếp con **theo chiều dọc**
Trục chính (mainAxis):
⬇️ Trên → Dưới
Dùng khi:
Màn hình
Form
Danh sách section

◈ mainAxis

Định nghĩa:
Trục **chính** của Row / Column

Widget	mainAxis
Row	Ngang
Column	Dọc

Lỗi tư duy phổ biến:
❌ mainAxis luôn là ngang

◈ crossAxis

Định nghĩa:
Trục **vuông góc** với mainAxis
Hiểu đơn giản:
👉 Trục "còn lại"

◈ mainAxisAlignment

Định nghĩa:
Cách **canh chỉnh widget theo trục chính**
Ví dụ:
spaceBetween
center
start / end

◈ crossAxisAlignment

Định nghĩa:
Cách **canh chỉnh widget theo trục phụ**
Lỗi sinh viên hay gặp:
❌ Căn giữa sai trục → layout lệch

III. THUẬT NGỮ KIỂM SOÁT KHÔNG GIAN

◈ Expanded

Định nghĩa:
Widget chiếm **toàn bộ không gian còn lại** trong Flex.
Hiểu đơn giản:
👉 "Cho mày chiếm hết phần còn dư"
Dùng khi:

PhucPT10@fpt.edu.vn

Fix overflow
Layout co giãn theo màn hình

◈ Flexible

Định nghĩa:
Widget **được phép giãn**, nhưng **không bắt buộc chiếm hết**
Khác Expanded:

Expanded	Flexible
Chiếm hết	Chiếm vừa đủ

◈ RenderFlex Overflow

Định nghĩa:
Lỗi layout khi **widget vượt quá không gian cho phép**
Dấu hiệu:
Sọc vàng / đen
Text tràn màn hình
Nguyên nhân:
Quên Expanded
Hard-code size

IV. THUẬT NGỮ VỀ GRIDVIEW

◈ GridView

Định nghĩa:
Widget hiển thị danh sách **dạng lưới có scroll**
Dùng khi:
Sản phẩm
Ảnh
Dashboard

◈ GridView.count

Định nghĩa:
Grid với **số cột cố định**
Dùng khi:
Dữ liệu ít
UI đơn giản

◈ GridView.builder

Định nghĩa:
Grid tạo item **theo nhu cầu (lazy loading)**
Dùng trong app thật:
👉 Danh sách lớn, dữ liệu động

◈ SliverGridDelegate

Định nghĩa:
Quy định **cách chia lưới**
Sinh viên thường bỏ qua → UI méo

V. THUẬT NGỮ VỀ STACK & UI CHỒNG LỚP

PhucPT10@fpt.edu.vn

◈ Stack
Định nghĩa:
Widget cho phép **chồng nhiều widget lên nhau**
Hiểu đơn giản:
👉 UI nhiều lớp (layer)

◈ Z-order
Định nghĩa:
Thứ tự **trước – sau** của widget trong Stack
Quy tắc:
Widget khai báo sau → nằm trên

◈ Positioned
Định nghĩa:
Widget xác định vị trí **chính xác** trong Stack
Dùng cho:
Badge
Icon overlay

VI. THUẬT NGỮ VỀ NAVIGATION

◈ Navigation (nội bộ)
Định nghĩa:
Điều hướng giữa các màn hình **bên trong app**
Ví dụ:
`Navigator.push`

◈ iOS URL Scheme
Định nghĩa:
Cơ chế cho phép **mở app từ link bên ngoài**
Ví dụ:
`myapp://products`

◈ Deep Linking
Định nghĩa:
Mở app và **đi thẳng vào màn hình cụ thể**
👉 URL Scheme là **nền tảng của Deep Linking**

◈ Route
Định nghĩa:
Đường dẫn điều hướng đến một màn hình
Hiểu đơn giản:
👉 Route = "địa chỉ" của screen

VII. THUẬT NGỮ VỀ STATE & CẤU TRÚC APP

◈ StatelessWidget
Định nghĩa:
Widget **không tự thay đổi trạng thái**

PhucPT10@fpt.edu.vn

◈ StatefulWidget

Định nghĩa:

Widget có **state thay đổi theo thời gian**

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

🔍 Prompt 1 – Tư duy Flex Layout trong Flutter

Phân tích vì sao Flutter chọn Flex-based layout (Row / Column) thay vì layout kiểu absolute positioning như nhiều framework UI khác.

Đánh giá ưu – nhược điểm của Flex layout khi xây dựng UI phức tạp và responsive trong app Flutter thực tế.

🎯 Mục tiêu tư duy:
Hiểu **bản chất thiết kế Flutter**
Chuẩn bị cho responsive UI

🔍 Prompt 2 – mainAxis / crossAxis và lỗi tư duy layout

Phân tích các lỗi UI phổ biến bắt nguồn từ việc hiểu sai mainAxis và crossAxis trong Row / Column.

Đánh giá vì sao các lỗi này thường không crash app nhưng gây UI xấu và khó maintain.

🎯 Mục tiêu:
Nhận diện lỗi "nguy hiểm nhưng im lặng"
Hiểu vì sao debug layout khó

🔍 Prompt 3 – Expanded vs Flexible trong dự án thật

So sánh Expanded và Flexible trong các tình huống UI thực tế (màn hình nhỏ, dữ liệu dài, đa ngôn ngữ).

Đánh giá hậu quả lâu dài nếu chọn sai widget kiểm soát không gian.

🎯 Mục tiêu:
Biết chọn đúng, không dùng theo thói quen
Tránh RenderFlex overflow về sau

🔍 Prompt 4 – Khi nào GridView là lựa chọn BẮT BUỘC

Phân tích vì sao việc tự dựng layout lưới bằng Column + Row là sai lầm trong app Flutter thực tế.

Đánh giá GridView.builder về hiệu năng, khả năng mở rộng và maintain so với grid thủ công.

🎯 Mục tiêu:
Hiểu GridView không chỉ là "cho đẹp"
Tư duy data-driven UI

🔍 Prompt 5 – Stack: công cụ hay bẫy?

Đánh giá Stack trong vai trò xây dựng UI chồng lớp: khi nào Stack là giải pháp đúng, khi nào là dấu hiệu thiết kế UI sai.

Phân tích rủi ro maintain khi lạm dụng Stack trong dự án lớn.

🎯 Mục tiêu:
Tránh "Stack everywhere"
Nhận biết code mùi (code smell) trong UI

🔍 Prompt 6 – Layout đúng nhưng UX sai

Phân tích các trường hợp UI layout đúng về mặt kỹ thuật (không lỗi) nhưng gây trải nghiệm người dùng kém.

Đánh giá vai trò của layout trong UX chứ không chỉ hiển thị.

🎯 Mục tiêu:
Liên kết layout ↔ UX
Bước đầu tư duy sản phẩm

PhucPT10@fpt.edu.vn

🔍 Prompt 7 – URL Scheme & tư duy navigation ngoài app

Phân tích sự khác biệt giữa navigation nội bộ trong Flutter và navigation từ hệ điều hành (iOS URL Scheme).

Đánh giá vì sao hiểu sai phần này khiến app khó tích hợp marketing, email, QR, deep link.

🎯 Mục tiêu:
Hiểu navigation ở mức hệ thống
Chuẩn bị cho deep linking

🔍 Prompt 8 – Layout và Widget Tree

Phân tích mối quan hệ giữa layout phức tạp và Widget Tree trong Flutter.

Đánh giá vì sao layout rối thường đi kèm widget tree khó đọc, khó debug.

🎯 Mục tiêu:
Hiểu layout không tách rời kiến trúc widget
Chuẩn bị cho refactor UI

🔍 Prompt 9 – Khi nào layout trở thành gánh nặng kỹ thuật

Đánh giá các dấu hiệu cho thấy layout Flutter đang trở thành technical debt.

Phân tích vai trò của tư duy layout đúng ngay từ Slot 9 trong việc giảm chi phí sửa UI về sau.

🎯 Mục tiêu:
Nhận diện technical debt sớm
Tư duy dài hạn cho dự án thật

🔍 Prompt 10 – Slot 9 trong lộ trình Flutter chuyên nghiệp

Đánh giá vai trò của kiến thức Slot 9 (Flex, GridView, Stack, URL Scheme) trong toàn bộ lộ trình Flutter từ beginner đến senior.

Phân tích điều gì sẽ xảy ra nếu lập trình viên bỏ qua hoặc học hời hợt các kiến thức này.

🎯 Mục tiêu:
Tự định vị kiến thức nền tảng
Nhận thức "học để làm lâu dài"

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

📦 NHÓM 1 – TƯ DUY LAYOUT TRONG DỰ ÁN QUY MÔ LỚN

🔍 Prompt 1 – Flex Layout và khả năng mở rộng UI

Phân tích cách Flex Layout (Row / Column / Expanded) ảnh hưởng đến khả năng mở rộng UI trong dự án Flutter lớn (nhiều màn hình, nhiều team cùng code).
Đánh giá hậu quả nếu layout được xây dựng theo kiểu "đủ chạy" từ giai đoạn MVP.

🔍 Prompt 2 – Khi layout trở thành technical debt

Nêu các dấu hiệu cho thấy layout Flutter đang trở thành technical debt trong dự án thực tế.
Phân tích vai trò của tư duy layout Slot 9 trong việc giảm chi phí refactor UI.

🔍 Prompt 3 – Widget Tree phình to và layout rối

Phân tích mối liên hệ giữa Flex Layout sai và Widget Tree phình to trong dự án lớn.
Đánh giá cách tổ chức widget giúp UI dễ đọc – dễ debug – dễ review code.

🔍 Prompt 4 – Chuẩn hóa layout trong team Flutter

Nghiên cứu cách các team Flutter lớn chuẩn hóa layout (guideline, reusable widgets, design system).
Đánh giá vì sao Slot 9 là nền tảng để áp dụng các chuẩn này.

📐 NHÓM 2 – GRIDVIEW & HIỂN THỊ DỮ LIỆU THỰC TẾ

🔍 Prompt 5 – GridView và dữ liệu thật

Phân tích các vấn đề phát sinh khi GridView hiển thị dữ liệu thật (nhiều item, hình ảnh lớn, dữ liệu động).
Đánh giá vì sao GridView.builder là lựa chọn gần như bắt buộc trong app thương mại.

🔍 Prompt 6 – Grid thủ công vs GridView

So sánh việc dựng layout lưới bằng Column/Row thủ công với GridView trong dự án lớn.
Phân tích chi phí bảo trì, hiệu năng và rủi ro bug UI.

🔍 Prompt 7 – Responsive Grid trong thực tế

Nghiên cứu cách xây dựng GridView responsive cho nhiều kích thước màn hình (mobile, tablet).
Đánh giá hạn chế của GridView.count và giải pháp thay thế.

🧩 NHÓM 3 – STACK & UI CHỒNG LỚP TRONG APP THẬT

🔍 Prompt 8 – Stack trong UI production

Phân tích vai trò thật sự của Stack trong UI production (badge, overlay, floating element).
Đánh giá ranh giới giữa "dùng đúng" và "lạm dụng Stack".

🔍 Prompt 9 – Rủi ro maintain khi lạm dụng Stack

Nghiên cứu các vấn đề maintain phát sinh khi Stack được dùng quá nhiều trong dự án lớn.
Phân tích vì sao Stack thường là nguyên nhân gây bug UI khó debug.

PhucPT10@fpt.edu.vn

🔍 Prompt 10 – Z-order & accessibility

Phân tích ảnh hưởng của z-order (thứ tự widget trong Stack) đến accessibility và trải nghiệm người dùng.
Đánh giá những rủi ro mà sinh viên thường bỏ qua.

🔀 NHÓM 4 – NAVIGATION & TÍCH HỢP HỆ ĐIỀU HÀNH

🔍 Prompt 11 – Navigation nội bộ vs hệ điều hành

So sánh navigation nội bộ Flutter với navigation từ hệ điều hành (iOS URL Scheme).
Phân tích vì sao dự án thương mại không thể chỉ dựa vào Navigator cơ bản.

🔍 Prompt 12 – URL Scheme & luồng người dùng thật

Phân tích các luồng người dùng thực tế sử dụng URL Scheme (email, QR, push notification).
Đánh giá yêu cầu layout & navigation khi app được mở từ nhiều entry point khác nhau.

🔍 Prompt 13 – URL Scheme và kiến trúc app

Nghiên cứu ảnh hưởng của URL Scheme đến kiến trúc Flutter app (routing, screen state).
Phân tích vì sao Slot 9 là bước chuẩn bị bắt buộc cho Deep Linking & Navigator nâng cao.

🧠 NHÓM 5 – TƯ DUY CHUYÊN NGHIỆP & QUY TRÌNH LÀM VIỆC

🔍 Prompt 14 – Layout & code review

Phân tích các tiêu chí đánh giá layout Flutter trong code review của team chuyên nghiệp.
Đánh giá vì sao layout sai thường bị reject dù app “chạy được”.

🔍 Prompt 15 – Slot 9 và năng lực Flutter Developer

Đánh giá vai trò của Slot 9 trong việc phân biệt Flutter Developer junior và mid-level.
Phân tích những kỹ năng layout & navigation mà nhà tuyển dụng kỳ vọng ở dự án thật.

PhucPT10@fpt.edu.vn